Embodied AI using Large Vision-Language Models and Vision-Language-Action Models

Prepared by:

Mohamed Amine AROUS
Diego FLEURY CORREA DE MORAES
Sergio MAGALHAES CONTENTE
Leonardo FALABELLA
Yasmin VAN DEN BROEK

Project:

P11 Embodied AI and LVLM, VLA

Academic Year:

2025/2026

Institution:

ENSTA Paris – Institut Polytechnique de Paris

Contents

1 Introduction **1**

2 Literature Review **3**
2.1 Zero-Shot Object-Goal Navigation with LVLMs 3
2.2 LVLMs as High-Level Planners and Scoring Modules 4
2.3 Vision-Language-Action (VLA) Architectures 4
2.4 Hierarchical Control Policies 5
2.5 Advanced Reasoning and Visual Search Efficiency 5

3 Methodology and Realizations **6**
3.1 Research Efforts 6
3.1.1 Navigation policies 6
3.1.2 Knowledge Graphs & Akinator 7
3.1.3 Extended Reasoning 7
3.2 Initial Experiments 8
3.2.1 Hardware Constraints and Dependency Issues 8
3.2.2 Baseline Evaluation and Diagnosis of the False-Positive Loop . 9
3.3 Approaches 10
3.3.1 Prompt Engineering and Pipeline Optimization 10
3.3.2 VLM Fine-Tuning for AIUTA-Compatible Responses 11
3.3.3 AIUTA-VLM-R1: Integrating RL-Trained VLMs with Knowledge Graphs 14

4 Results **17**
4.1 Prompt Engineering 17
4.2 COCO-Based + IDKVQA VLM Fine-Tuning 17
4.3 COCO + Augmented IDKVQA VLM QLora Fine Tuning 18
4.4 AIUTA-VLM-R1 19
4.4.1 Offline Uncertainty Benchmarking 19
4.4.2 Quantitative Results - VLM-R1 20

5 Discussions and Conclusion **21**
5.1 Prompt Engineering 21
5.2 VLM Finetuning 21
5.3 VLM-R1 and Knowledge Graph Integration 22

6 Contributions **23**

A Study of the CoIN Framework and the AIUTA System **24**
A.1 Formal Definition of State Variables and Prompts 24
A.2 The AIUTA Dataflow and Uncertainty Estimation 25
A.3 Information-Theoretic Interpretation and Novelty 27
A.4 Codebase Architecture and Dependency Graph 27
A.5 Cluster Deployment and Technical Setup 27
A.5.1 Metric Aggregation Failures and System Stability 28
A.5.2 Evaluation Outcomes and the Over-Pruning Effect 28

B AIUTA Dataflow Diagrams **30**

C VLM Finetuning experiments results **34**

D AIUTA-VLM-R1: Architechture and Dataflow diagrams **37**

List of Figures

3.1 Label distribution before and after augmentation in the IDKVQA training set. 13

4.1 Confusion matrices for baseline and fine-tuned models. 19

B.1 Initialization phase and triggering of the AIUTA module. 30
B.2 Self-questioner data flow used for semantic enrichment. 31
B.3 Uncertainty estimation module based on normalized Shannon entropy (core innovation). 32
B.4 Interaction trigger data flow and human-in-the-loop update mechanism. 33

C.1 Training dynamics of the QLoRA + COCO-mix pipeline (loss and validation accuracy). 34
C.2 Per-class accuracy comparison between baseline and fine-tuned models. 35
C.3 Per-class F1-score comparison between baseline and fine-tuned models. 35
C.4 Qualitative prediction examples from the fine-tuned model. 36

D.1 AIUTA-VLM-R1 system overview. 37
D.2 AIUTA-VLM-R1 reasoning pipeline. 38
D.3 Knowledge graph interaction flow with reasoning model's output. . . 39
D.4 VLM/LLM calls compairison between original AIUTA and proposed AIUTA-VLM-R1. 40

Chapter 1

Introduction

Embodied Artificial Intelligence is a field at the intersection of robotics, computer vision, and machine learning, aiming to design agents capable of perceiving, reasoning, and acting in dynamic environments. Unlike traditional AI systems operating on static datasets, these agents must make decisions under incomplete and evolving observations, which introduces challenges in tasks such as navigation and object recognition. Recent advances in multimodal foundation models - particularly **Large Vision-Language Models (LVLMs)** (as in [25] and [8]) and **Vision-Language-Action (VLA) models** (as in [4] and [20]) - have significantly enhanced these capabilities by enabling semantic reasoning from visual inputs and bridging perception with action. However, integrating such models into robust embodied systems remains difficult, especially when dealing with uncertainty, instance-level reasoning, and reliable interaction with users.

In this context, this project focuses on the study and improvement of the **CoIN (Collaborative Instance Navigation)** [16] framework and its core component, the **AIUTA system**. Unlike standard navigation approaches that operate at the category level (e.g., finding any "chair"), CoIN introduces instance-level reasoning, where the agent must identify a specific object described by detailed attributes. AIUTA plays a central role in this process as a semantic verification and interaction module that is triggered during navigation. It operates on top of a VLFM-based navigation loop that : *explores* the environment, *detects* candidate objects, and delegates *semantic reasoning* to AIUTA. The system relies on structured state variables and prompt-driven interactions, where visual observations are progressively transformed into increasingly refined semantic representations through a sequence of vision-language and language-only reasoning steps.

At a high level, the AIUTA pipeline follows a staged reasoning process. When a candidate (defined through its current "fact state" F_t) object is detected (observation O_t), a vision-language model first generates an initial description of the scene (S_{init}). This description is then enriched through a self-questioning mechanism, where a language model proposes follow-up questions and the vision model answers them, producing a more detailed semantic representation ($S_{enriched}$). The system then introduces an uncertainty-aware decision layer by querying the model with constrained answers (Yes/No/I don't know) and computing a normalized entropy score over the output distribution (u_{check}). This uncertainty measure determines

whether the system should proceed, discard the observation, or request further information. If the prediction is deemed reliable ($u_{check} < \tau$), a second level of reasoning extracts object attributes and verifies them individually, filtering out uncertain and ambiguous elements to produce a refined and trustworthy description ($S_{refined}$). This representation is used to compute an alignment score s with the target and, if needed, generate a clarification question $q_{a \to u}$, updating the fact state over time (producing, in case of failure, new facts F_{t+1} to be stored for later use). Finally, if the system evaluated that the identified object matches the target instance, it halts.

From an architectural perspective, AIUTA can be interpreted as a sequence of structured transformations operating over multimodal inputs and symbolic representations:

$$(F_t, O_t) \to S_{\text{init}} \to S_{\text{enriched}} \to u_{\text{check}} \to S_{\text{refined}} \to (s, q_{a \to u}) \to F_{t+1}.$$

The primary novelty of AIUTA lies in its **uncertainty-gated control flow** and **attribute-level uncertainty filtering**. By explicitly quantifying epistemic uncertainty and pruning unreliable attributes, the system converts raw vision-language outputs into a structured and verifiable representation, enabling targeted and context-aware interactions and improving robustness in ambiguous scenarios.

The **objective of this project** is to design, analyze, and improve an embodied AI pipeline capable of **instance-level navigation under uncertainty**. More specifically, this work aims to:

- understand and reproduce the CoIN/AIUTA framework;
- analyze its internal mechanisms and identify key limitations;
- improve its reasoning and decision-making capabilities through prompt engineering and system-level modifications;
- evaluate the impact of these improvements using standardized benchmarks.

The remainder of this report is organized as follows. Chapter 2 reviews related work on embodied AI and vision-language models. Chapter 3 presents the methodology and experimental setup. Chapter 4 reports the results and evaluation. Finally, Chapter 5 provides a critical analysis and discusses future directions.

Chapter 2

Literature Review

The rapid advancement of multimodal foundation models [21, 24, 29] has profoundly impacted Embodied AI. Recent literature demonstrates a clear trajectory from modular, semantic-mapping approaches to end-to-end, action-generating models. This section reviews the state-of-the-art across two main paradigms: Large Vision-Language Models (LVLMs) used for zero-shot navigation, and Vision-Language-Action Models (VLAs) used for direct or hierarchical control.

2.1 Zero-Shot Object-Goal Navigation with LVLMs

A significant focus in recent robotics research is Zero-Shot Object-Goal Navigation (ObjectNav), where an agent must locate a specified object in an unexplored environment without prior task-specific training. A leading approach in this domain is VLFM (Vision-Language Frontier Maps) [35]. As detailed in the literature, VLFM achieves state-of-the-art zero-shot navigation by utilizing a pre-trained LVLM (BLIP-2) [21] to evaluate the semantic value of unexplored areas (frontiers). Operating in three phases-initialization, exploration, and goal navigation-VLFM constructs an occupancy map using depth sensors and queries the LVLM using RGB observations to assign a "semantic score" to different frontiers. Once the robot navigates to a high-value area, it switches to an object detector (e.g., YOLO or GroundingDINO) for the final approach. While highly successful in simulators like Habitat and on real-world robots like Boston Dynamics' Spot, VLFM suffers from a "one-track mind" (maps are task-specific) and the "height problem," assuming all objects are visible from a standing posture [35].

Contrasting with VLFM's modularity is the ZSON (Zero-Shot Object-Goal Navigation) framework [27]. ZSON decouples semantic goal representation from navigation learning. It utilizes a multimodal embedding space (CLIP) [29] to bridge image goals and language goals. The agent is trained extensively on a proxy task-Image-Goal Navigation (ImageNav)-using reinforcement learning (RL). At inference, the text goal is projected into the CLIP embedding space, allowing the recurrent policy to navigate end-to-end using only RGB sensors. While ZSON features a simpler inference architecture and fewer sensor dependencies than VLFM, it significantly underperforms in complex environments due to the inefficiency of recurrent "memory" exploration versus explicit semantic mapping [27].

To address the unreliability of semantic cues in novel environments-such as facing a blank wall or ambiguous objects-ApexNav introduces an adaptive exploration strategy [37]. It dynamically analyzes the semantic distribution of the environment. When semantic cues are strong, it leverages LLM-driven reasoning to guide the agent; when cues are weak, it seamlessly switches to geometry-based exploration to rapidly expand map coverage. Furthermore, ApexNav implements a target-centric semantic fusion mechanism, aggregating multi-frame observations to maintain a reliable long-term memory of target objects, mitigating the noise of imperfect object detectors [37].

Similarly, the CoW (CLIP on Wheels) framework [6] demonstrates that pre-existing vision models can be used effectively without fine-tuning. CoW uses CLIP [29] to score camera views against text descriptions while relying on simple Frontier-Based Exploration (FBE). Interestingly, CoW reveals that simple, logical exploration combined with strong open-vocabulary recognition often matches the efficiency of models trained via millions of RL steps [6].

2.2 LVLMs as High-Level Planners and Scoring Modules

Beyond pure exploration, LVLMs are increasingly used to generate constraints and code for existing navigation stacks. The VLM-Social-Nav framework [7] utilizes GPT-4V to infer social context from current images and robot states. Instead of generating actions directly, the VLM outputs a text behavior which a scoring module converts into a "social cost" term. This cost is fed into a traditional optimization-based planner, improving social compliance and eliminating the need for large-scale social training datasets [7].

Similarly, research into VLM-based human-guided mobile robot navigation [22] demonstrates how LLMs can interpret user voice commands and few-shot examples to generate executable Python code. This code interfaces with a pre-programmed navigation system operating on a detailed 3D map, effectively utilizing the LLM as a semantic bridge between human intent and rigid robotic APIs [22].

2.3 Vision-Language-Action (VLA) Architectures

While LVLMs excel at reasoning, they require complex engineering to translate text into physical movement. VLA models attempt to solve this by generating actions directly. Models like ChatVLA [32] address the challenges of real-world robotic manipulation. ChatVLA utilizes a "Mixture-of-Experts" (MoE) architecture, separating a vision-language understanding expert from a control expert. Utilizing a novel "Phased Alignment" training method, it prevents the catastrophic forgetting often seen when fine-tuning VLMs for robotics, achieving superior success rates on real-world arm tasks compared to strong baselines [32].

Recent work such as RT-2 [4] and PaLM-E [12] further demonstrates the potential of end-to-end vision-language-action models for embodied reasoning and control.

Its successor, ChatVLA-2 [2], focuses on open-world robotic tasks requiring deep reasoning, such as math matching games and novel object placement. It employs a dynamic MoE system where a dedicated "action expert" generates movements guided by a "reasoning following module." This ensures that the robot's physical actions strictly align with the VLM's internal logical deductions, outperforming previous VLAs in generalization to unseen problems [2].

2.4 Hierarchical Control Policies

A growing consensus in Embodied AI is the effectiveness of hierarchical systems that combine the reasoning power of VLMs with the reactive safety of low-level policies. The NaVILA framework [9] demonstrates this on legged robots. It employs a high-level VLA as the "brain," which processes user commands and visual history to output mid-level text instructions. A separate, low-level RL policy then translates this text into joint motor actions. This separation allows the brain to be trained on diverse internet-scale data while the low-level policy remains robot-specific [9].

The Hi-Robot architecture [31] pushes this further for complex, open-ended tasks like grocery shopping or making sandwiches. It functions on a "System 2 / System 1" cognitive model. A high-level VLM acts as System 2, interpreting complex requests and live visual feedback to generate simple, step-by-step text commands. A low-level VLA policy (π_0) acts as System 1, taking these simple commands alongside live camera feeds to generate continuous robot movements. This hierarchy achieves near-human performance in generating logical sub-goals and handling real-time corrections [31].

2.5 Advanced Reasoning and Visual Search Efficiency

Finally, the efficiency of the foundation models themselves remains a critical area of research. Models like VLM-R1 [30] focus on providing stable and generalizable reasoning architectures for vision-language tasks. Furthermore, techniques like Adaptive Chain-of-Focus (CoF) [38] introduce dynamic visual search and zooming for VLMs. These approaches are closely related to recent advances in Chain-of-Thought reasoning [33]. Instead of processing entire high-resolution images simultaneously-which is computationally expensive for real-time robotics-CoF allows the model to dynamically focus on and zoom into relevant parts of the scene based on the task, drastically improving inference efficiency without sacrificing reasoning quality [38].

Chapter 3

Methodology and Realizations

The first part of our efforts were mainly directed towards the goal of research, trying to get a hold of similar architectures, ideas, and inspire/educate ourselves on the subject of Instance Object Navigation, alongside the entire context of embodied AI. The 3 main research directions were **changing the navigation policy**, **improving instance reasoning via Knowledge Graphs** and **improving performance via extended reasoning capabilities**.

Following that, with our findings (and access to the Mesogip cluster) we began performing 2 optimizations to the pipeline (prompt engineering and finetuning over the desired behavior), following what our results were telling us, alongside a different proposal of the AIUTA ecossystem, to overcome compute limitations and explore a different direction.

3.1 Research Efforts

To structure our investigation, we focused on three main directions: **changing the navigation policy**, **improving instance reasoning via Knowledge Graphs**, and **improving performance via extended reasoning capabilities**.

3.1.1 Navigation policies

AIUTA is not a complete navigation system, but a triggered semantic verification module that activates once an object candidate has been detected during spatial exploration. The upstream VLFM [35] loop builds a frontier map, assigns semantic value to unexplored regions, and uses Grounding-DINO [26] for object detection. The global control logic maintains a strict separation of concerns: the navigation stack answers *where to go next*, while AIUTA answers *what did we just find, and when should we ask the human*.

Integrating AIUTA requires a modular navigation policy that exposes an explicit object detector to trigger the reasoning layer. We evaluated several candidates: **TriHelper** [36] was conceptually aligned but lacked an open-source implementation; **OpenVLA** [19] targets manipulation rather than navigation; **GAMap** [14] offered promising attribute-based mapping but was architecturally complex to integrate;

and **CoWs on Pasture** [6] relied on CLIP heatmaps without discrete detections, requiring a redundant parallel detector.

OpenFMNav [15] appeared most compatible, as its vision module produces language-labeled detections that match AIUTA's input format directly. However, further analysis revealed that all evaluated candidates - OpenFMNav, VLFM, GAMap, and L3MVN [13] - share the same core mechanism: map-based frontier exploration for category-level navigation. Since the primary challenge lies in *disambiguation and verification* rather than exploration, replacing VLFM with another frontier-based method would add complexity with limited benefit. We concluded that efforts were better directed elsewhere.

3.1.2 Knowledge Graphs & Akinator

Two papers on prototypical learning informed the reasoning layer: Proto-CLIP [28], which aligns image and text prototypes via contrastive learning for robust few-shot classification, and "Right for the Right Reasons" [1], which anchors neural embeddings to labelled prototypes to prevent spurious concept associations. Both share the intuition that anchoring examples can hard-constrain an open-ended embedding space. This motivated an **Akinator-inspired direction**: rather than freely associating observations with target descriptions, the system would progressively **narrow the hypothesis space** through structured binary questions - reducing open-endedness while lowering computational cost and enabling task reuse. These ideas directly inspired our later implementations.

3.1.3 Extended Reasoning

Two works informed our use of structured reasoning traces: VLM-R1 [30] and Adaptive Chain-of-Focus (Adaptive-CoF) [38].

VLM-R1

VLM-R1 [30] shows that R1-style reinforcement learning with GRPO [17] substantially improves visual reasoning on tasks like Open-Vocabulary Object Detection (OVD). RL-trained models generalize significantly better than SFT counterparts out-of-domain, and the training naturally produces `<think>` blocks containing structured natural-language reasoning about object presence, attributes, and spatial context. We exploit this directly: the VLM-R1 OVD checkpoint's reasoning traces are parsed via regex (TripleExtractor) to populate our Knowledge Graph, converting the model's internal deliberation into structured, queryable facts at zero additional cost.

Adaptive Chain-of-Focus

Adaptive-CoF [38] teaches VLMs to zoom into image regions *only when necessary*, avoiding the token overhead and context loss of constant zooming. It combines SFT on multi-step reasoning trajectories with an Adaptive Group-Aware Reward

(AGAR) that incentivizes direct answers when sufficient visual information is available. On the V* benchmark [34], it reduces zoom operations by 75% with nearly 50% fewer visual tokens. This adaptive philosophy directly influenced our Interaction Trigger design: the GraphMatcher scores alignment between detected attributes and target facts, deciding whether to ask the user, continue exploring, or stop - skipping the Oracle entirely when the KG already contains sufficient negative evidence.

3.2 Initial Experiments

To conduct these experiments, we first addressed key hardware and dependency constraints that shaped our setup.

3.2.1 Hardware Constraints and Dependency Issues

GPU job allocation

Following the decision to pivot our focus toward enhancing the interaction layer, our next experimental phase aimed to run the CoIN-Bench evaluation (`val_unseen` split) to benchmark different LLM models.

Initially, we utilized the Groq API running the **Llama-3.3-70B** model. However, because the AIUTA dataflow requires multiple LLM calls per detection (for question generation, attribute extraction, and scoring), performing just 10 evaluation iterations rapidly exceeded the daily free-tier rate limits, causing the evaluation pipeline to crash !

To ensure reproducibility and bypass API constraints, we transitioned to hosting LLM models locally on the computing cluster. This introduced strict **hardware resource constraints**. The available cluster node contained four NVIDIA H100 GPUs. Because running a **70B parameter** model locally requires massive VRAM overhead, we downscaled our target LLM to an **8B parameter** model (**Llama-3-8B**) and parallelized the workload.

The constrained GPU allocation was designed as follows:

- **GPU 0:** Detection stack (GroundingDINO, MobileSAM, BLIP-2/LAVIS).
- **GPU 1:** Vision-Language Model server (LLaVA-NeXT).
- **GPU 2:** Evaluation and navigation loop (Habitat-Sim / CoIN-Bench).
- **GPU 3:** Local LLM server (running via `vLLM`).

Environment Isolation

Deploying the multi-server pipeline quickly exposed a number of dependency issues in the original CoIN setup. During evaluation runs, the launcher would often fail

with the error: *"ERROR: One or more VLM servers did not open ports in time."* This suggested that some components were crashing silently during initialization.

After investigating the logs, we identified several conflicts that made it impossible to run the full system within a single Python environment. First, Habitat-Sim depends on `numpy-quaternion`, which is compiled against NumPy 1.x. Since our environment had `numpy==2.0.2`, this caused immediate crashes, forcing us to downgrade to `numpy==1.26.4`. Fixing this issue revealed another one: the BLIP-2 module from `salesforce-lavis==1.0.2` requires an older version of Transformers (below 4.27). However, switching to this older version broke the LLaVA-NeXT server, which depends on more recent versions of Transformers (above 4.40) due to the `LlavaNextProcessor` class.

At this point, it became clear that these components were fundamentally incompatible within a single environment. In practice, LAVIS and LLaVA-NeXT could not be made to work together without isolating their dependencies.

To address this, we restructured the system by splitting it into **multiple Conda environments**, taking advantage of the fact that the CoIN pipeline runs as separate processes communicating over HTTP. We created **three environments**: one dedicated to the **vLLM server**, one for the **core navigation and detection stack** (including Habitat, GroundingDINO, SAM, and LAVIS), and one specifically for the **LLaVA-NeXT server** with its newer dependencies.

Finally, we modified the startup scripts (see *run_coin_eval_final.sh*) so that each service runs within its own environment. This approach resolved the conflicts entirely and allowed the system to run reliably.

3.2.2 Baseline Evaluation and Diagnosis of the False-Positive Loop

With the system stabilized locally, we ran the baseline evaluations. The quantitative diagnostics immediately revealed a severe behavioral collapse. While the metrics for `target_detected` (~ 0.60) and `stop_called` (~ 0.50) were exceptionally high, the actual success rate was disastrously low (~ 0.10). The agent exhibited a near 100% failure rate across episodes, terminating primarily due to false-positive errors !

Logs indicated that the robot would repeatedly detect a single incorrect class (e.g., a distractor "dresser"), navigate to it, falsely confirm it was the target, and prematurely end the episode.

Investigating Navigation Logic (Instance Collapse and Confirmation Bias)

Our initial hypothesis was a failure in the robotic navigation logic. Analyzing `ObjectPointCloudMap.get_best_object` revealed a phenomenon we termed *Instance Collapse*. When the agent detected multiple instances of a class (e.g., three different dressers in a room), the code merged them into a single point cloud array. Furthermore, the navigation goal selection relied on pure geometry (nearest point), ignoring the **semantic similarity scores** until a fallback trigger at step 400.

While this geometrical "nearest-point" navigation was inefficient, further log analysis revealed it was **merely a symptom**. The true failure was upstream in the

cognitive reasoning layer: we developed a log-analysis script to extract similarity scores, VLM answers, and question counts. The results revealed a clear issue in the scoring behavior.

First, **similarity scores were too high overall**: more than 72.7% of detections received a score ≥ 7, which *frequently* triggered stop decisions for incorrect objects. Second, the VLM showed a **strong confirmation bias**, with around 63% "Yes" answers, compared to 24% "No" and 13% "Uncertain". Finally, even when obvious mismatches were present (e.g., a cardboard dresser instead of a wooden one), the LLM still assigned high scores as long as the general object category matched.

In practice, the system was not distinguishing between correct and incorrect instances, but only between object categories. This made the threshold $\tau_{\text{stop}} = 7$ ineffective, since many false positives were still accepted.

Addressing this required changing **how** similarity was evaluated, rather than simply tuning thresholds or modifying the navigation logic. This led us to two approaches : redesigning the prompting strategy and finetuning the LVM towards the desired answers ("Yes, No and I Don't Know"). Other adjustments to the AIUTA reasoning pipeline also had to be made, as described in the next section.

3.3 Approaches

To address the limitations identified during the experimental phase, we then focused on refining the reasoning component of the system, which became the primary bottleneck once deployment and navigation were stabilized.

3.3.1 Prompt Engineering and Pipeline Optimization

After stabilizing the system, the main limitation was no longer in deployment or navigation, but in the reasoning layer. Because of hardware constraints, we had to rely on smaller local language models (`Llama-3-8B`, `Qwen2.5-Coder-32B-Instruct`, and `GPT-oss-20B`) instead of the larger commercial models used in the original CoIN setup. In practice, these smaller models were less reliable for strict instance-level discrimination, which led to many false positives during evaluation.

This made prompt engineering a central part of the work. Rather than adjusting thresholds, we needed to change how the system represents objects, handles uncertainty, and evaluates semantic similarity. We therefore redesigned several prompts in `vlfm/utils/prompts.py` to make the reasoning process stricter and more consistent.

Epistemic Re-Engineering of the Prompts

The main system logic and entropy-based uncertainty calculation were already in place, but the prompt templates were not well aligned with the instance-matching task. We modified them in three ways.

First, we addressed the VLM's tendency to overproduce positive answers. The original Yes/No questions were often too simple, which encouraged confirmation

bias. To reduce this effect, we changed the self-questioning prompt so that it generated contrastive questions, for example by checking both whether an object was white and whether it was brown or wooden. This made ambiguous cases easier to detect and increased the entropy when the evidence was weak.

Second, we changed the refinement stage. Previously, uncertain information was simply removed, which often produced descriptions that were too generic and hid useful warning signs. We modified this step so that uncertainty is preserved explicitly in the refined representation, allowing ambiguity to remain visible in later stages of the pipeline.

Third, we made the similarity scoring rule stricter. The previous prompt allowed partial credit even when important attributes were wrong. We replaced this with a stricter rule: if a primary attribute such as color, material, or geometry contradicted the target description, the similarity score had to remain low. When uncertainty remained, the system was encouraged to generate a clarification question instead of accepting the object too early.

Because the hardware constraints discussed in Chapter 3 forced the use of a smaller, local Large Language Model (Llama-3-8B / Qwen2.5-Coder-32B-Instruct / GPT-oss-20B) rather than the massive commercial models (e.g., GPT-4o) originally utilized in the CoIN paper, the system lacked the implicit zero-shot reasoning required for strict instance-level target discrimination. Consequently, optimizing the cognitive reasoning layer through advanced prompt engineering and pipeline debugging became the critical path of our methodology.

The Autoregressive Chain-of-Thought Trap

Despite tightening the prompts, false positives persisted due to a fundamental architectural issue in how the Python backend parsed LLM outputs.

Originally, the LLM was instructed to output structured YAML first, followed by reasoning. However, Large Language Models generate tokens autoregressively (left-to-right), meaning the model was forced to produce the YAML before completing its reasoning. As a result, it hallucinated attributes in the YAML block.

Even when the reasoning later corrected these hallucinations, the backend extracted only the YAML block using:

```
response.split("YAML_START")[1].split("YAML_END")[0]
```

In order to fix this, we inverted the output structure : the LLM now generates reasoning first (Chain-of-Thought), followed by the YAML block. This enforces a sequential reasoning pipeline and ensures that the extracted structured output is consistent with the model's final reasoning.

3.3.2 VLM Fine-Tuning for AIUTA-Compatible Responses

Another approach taken, for minimizing the confirmation bias presented earlier on, was to focus on fine-tuning the VLM so that its responses aligned more closely with the AIUTA decision logic. The objective was to achieve a finer, more robust, calibration on the constrained answer space ({Yes, No, I don't know}). This tackles

more directly the **overclaim** problem as a whole (when the model engages in more assertive "Yes/No" answers for cases in which it should've said "IDK") which is required by the uncertainty-aware trigger mechanism in CoIN/AIUTA. Two strategies were employed with this goal in mind : mixing the **COCO dataset [23] with IDKVQA [3]** and using **data-augmented IDKVQA** with COCO. We also tried using another dataset from real life images called "**HouseObj_VQA**" [5] alongside IDKVQA, but results didn't really improve by comparison, so it's not further detailed here.

For both setups, beyond argmax accuracy, the CoIN-style reliability logic was reproduced by computing normalized entropy over the three-class probabilities and sweeping a threshold $\tau \in [0, 1)$ with step 0.01. When entropy exceeded τ, the output was forced to *I don't know*. For each checkpoint, the best τ was selected according to Effective Reliability (ER), matching the evaluation protocol used in the AIUTA pipeline.

COCO + IDKVQA

Motivation and Training Strategy

The IDKVQA dataset is task-aligned but relatively small; consequently, training exclusively on this dataset increases the risk of overfitting. To improve generalization while preserving task relevance, a mixed-data strategy was initially adopted.

VQAv2 samples were converted into three classes: *Yes*, *No*, and *I don't know*. All non-yes/no answer types (e.g., *number*, *other*) were mapped to *I don't know* to encourage explicit abstention when a binary decision is not justified.

A local IDKVQA-format ground-truth file and image structure were generated from the public split. Train/validation/test partitions were then defined at the **image** level (rather than at the QA-pair level) to prevent data leakage. The final split was 60% train, 20% validation, and 20% test. During training, an 80/20 mixed dataset was used:

- 80% COCO-derived VQA samples;
- 20% IDKVQA train samples (repeated when needed to preserve ratio).

We trained on this mixture, in order to reduce domain shift while maintaining optimization pressure on the target task, while validating and testing exclusively on the IDKVQA dataset.

Model Adaptation and Optimization

As the base model (`llava-hf/llava-v1.6-mistral-7b-hf`) had sufficient parameters to render full finetuning impractical (with the risk of overfitting), we aimed at maintaining computational feasibility at a maximum, applying LoRA [18] adapters to attention projection layers (`q_proj`, `v_proj`). Training used gradient accumulation to emulate a larger effective batch size, AdamW optimization, cosine learning-rate scheduling with warmup, gradient clipping, and early stopping based on validation accuracy.

As mentioned, the training loss was designed to align with constrained-answer inference. For each sample, logits were projected onto the three answer tokens

(Yes/No/?), normalized, and optimized using soft-label cross-entropy (on these 3 possibilities, indiscriminately). We initially tried to also apply a sparsity normalization penalty to all the other logits (penalizing heavy tails) but as training remained unstable, while barely improving, we chose to abandon this idea.

Its important to point out that soft labels were particularly useful for IDKVQA, due to the facts that even the annotators' votes were uncertain on that matter. Converting them into probabilistic targets rather than hard labels was only logical as well.

Models were evaluated on the IDKVQA validation and test splits, comparing the baseline (non-fine-tuned LLaVA) and the fine-tuned checkpoints obtained from the hyperparameter search.

QLoRA Augmented IDKVQA + COCO

Our second fine-tuning pipeline, implemented in `finetune_llava_idkvqa.py`, combines QLoRA [11] training, stronger data augmentation (with techniques gathered from [10]), and a broader evaluation suite (accuracy, per-class metrics, confusion matrices, and qualitative inspection). Soft labeling, validation set monitoring and data leakage prevention scheme followed the exact same procedure as before - as well as the optimizing procedure. Our training setup was the following :

- **Backbone and adaptation strategy:** `llava-hf/llava-v1.6-mistral-7b-hf` fine-tuned with QLoRA (4-bit quantization + LoRA adapters), reducing memory cost while retaining adaptation capacity.
- **Augmentation strategy:** 33 deterministic image transforms combined with 4 text templates per label. This generated substantial diversity in visual style and linguistic phrasing.
- **Mixed-data training:** COCO VQA [23] and the augmented IDKVQA were combined to improve generalization and reduce overfitting to the small target dataset. Again, testing and validation were only performed in the IDKVQA.

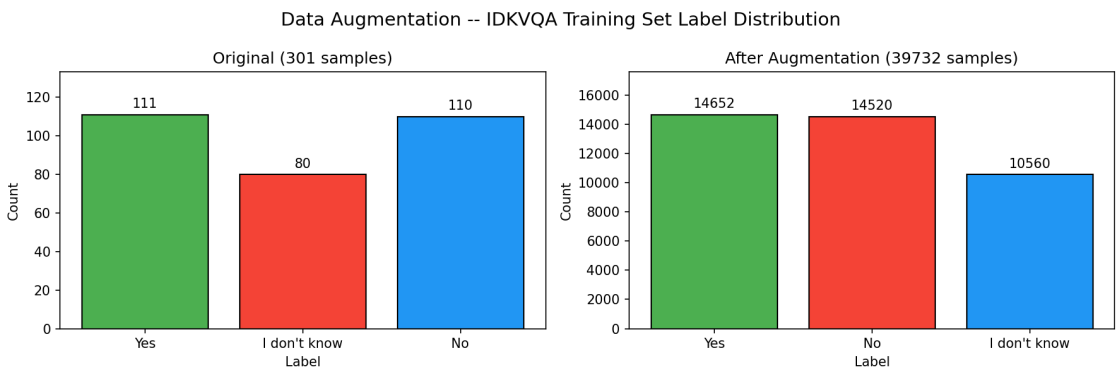

Figure 3.1: Label distribution before and after augmentation in the IDKVQA training set.

The augmentation process expanded the original 301 training samples to 39,732 augmented samples, increasing diversity while maintaining class coverage.

3.3.3 AIUTA-VLM-R1: Integrating RL-Trained VLMs with Knowledge Graphs

Description

The original AIUTA pipeline combines multiple specialized models: Grounding-DINO [26] for object detection, LLaVA 1.6 for visual description, and GPT-4o for question generation, fact management, and alignment scoring. Each detected object triggers 5-8 sequential VLM/LLM calls, creating a pipeline that is both **slow and expensive**. This section investigates whether a single RL-trained Vision-Language Model - the previously studied **VLM-R1** (Qwen2.5-VL-3B, trained with GRPO on the D3 dataset) - can replace this multi-model stack while maintaining or improving reasoning quality. VLM-R1's chain-of-thought reasoning, learned through reinforcement learning and expressed in '<think>' blocks, already contains the scene understanding, attribute extraction, and absence reasoning that AIUTA distributes across multiple models. To provide the structured memory that AIUTA's concatenated text facts lack, we augment VLM-R1 with a **per-episode Knowledge Graph** that accumulates object attributes across timesteps and enables **deterministic alignment scoring** without LLM calls.

Core contributions

First, we demonstrate that a 3B parameter RL-trained VLM can achieve competitive uncertainty calibration on the IDKVQA benchmark ($\phi_{binary} = 21.71$), comparable to the AIUTA baseline ($\phi_{binary} = 21.12$), **while using a 2-3 model calls instead of 5-8**. Second, we design and implement a **complete Knowledge Graph-based reasoning pipeline** that replaces AIUTA's LLM-driven question generation, fact management, and alignment scoring with deterministic graph operations, **reducing per-detection model calls from 5-8 to 2-3** (see D.4). Third, we integrate this pipeline with the VLFM frontier exploration framework for online evaluation on CoIN-Bench, **demonstrating correct multi-instance disambiguation** through attribute contradiction detection and positive fact confirmation.

System overview

The final system is summarized in Figs. D.1 and D.2. It operates as follows : the VLFM frontier explorer operates the same way as in CoIN with BLIP-2 and MobileSAM. When it detects an object matching the target category, the **VLM-R1 detector runs a single forward pass** producing both *bounding boxes* (in the '<answer>' block) and *chain-of-thought reasoning* (in the '<think>' block). A detection filter removes oversized, tiny, or malformed bounding boxes. The surviving detections are cropped from the observation frame and passed to the **SelfQuestioner**, which runs one additional VLM call to generate a rich feature description of the crop. Both the '<think>' triples and the description features are parsed into **canonical attribute names** (color, material, size, near) and stored in the **scene Knowledge Graph**.

The **QuestionGenerator** then examines the detected object's attributes, infers their semantic type (recognizing that "green" is a color, "wood" is a material), and

generates a *targeted* yes/no question using the appropriate template. The **Oracle** (a VLM-R1 instance viewing the target's instance image) answers with yes or no. The response is parsed - with the attribute value extracted from the question and the polarity from the response - and stored as a positive or negative target fact. The **GraphMatcher** computes alignment between target facts and object attributes, requiring at least one positive fact match before emitting STOP. If the Oracle contradicts the Detection VQA (Oracle says "no" while the detection crop says "yes"), the pipeline immediately emits CONTINUE as a definitive mismatch. (See D.3).

Architecture Design and Implementation

The architecture follows three principles: **minimize model calls** (replace LLM reasoning with deterministic KG operations), **maximize structure** (replace concatenated text facts with typed attribute triples), and **maintain "swappability"** (Strategy pattern for all components, allowing baseline comparison).

The system is composed of four modules: a **detector** that extracts and filters object regions, a **self-questioner** that derives structured attributes and canonical representations from model outputs, an **interaction trigger** that scores alignment between detected objects and target facts to decide actions, and a **knowledge graph** that maintains object states and accumulates positive and negative attributes.

A key design shift replaces open-ended queries with yes/no questions, enabling **deterministic parsing** and comparison without additional model calls. Detected attributes are converted into canonical forms, used to generate binary questions, and updated in the knowledge graph based on Oracle responses, allowing direct contradiction or confirmation checks.

To ensure consistency *across* heterogeneous attribute sources, a value-based semantic inference system maps extracted values to **canonical attribute types**, defined in three frozen sets of vocabulary (e.g., colors, materials, sizes). This guarantees reliable matching and alignment regardless of how attributes are initially expressed.

Several other modifications were needed to make all of this work, on pretty much all levels of the original implementation, ranging from timing, to detection to initialization schemes. These are not cited here for conciseness purposes.

Offline Uncertainty Benchmarking (IDKVQA)

Before integrating with the full CoIN navigation stack, which requires significant engineering effort, we needed to validate a fundamental question: *can VLM-R1, an OVD-trained model, handle uncertainty-aware visual question answering?* The IDKVQA dataset, in this context (again), provides the ideal testbed. The VLM-R1 model processes the image and question in a single forward pass, and we measure both the answer and the model's uncertainty (via token-level entropy).

We implemented eight evaluation modes, each adding a component to the pipeline:

- **Raw**: Direct VLM-R1 VQA. The model sees the image and question and outputs an answer. No gating, no KG, no additional processing. Measures the zero-overhead baseline for intrinsic VQA capability.
- **Threshold**: Same as Raw, but answers with first-token entropy above a threshold τ are converted to "I don't know." Tests whether the token probability distribution carries useful uncertainty signal. τ is swept from 0.05 to 0.35.
- **KG**: Runs OVD detection, extracts attributes via TripleExtractor into a per-image KG, then answers via KG attribute matching. If the KG lacks the relevant attribute, abstains.
- **Two-pass KG**: OVD detection plus a second VLM call requesting structured JSON attributes (color, material, size, features, spatial, exists). Provides richer attributes at the cost of an additional call.
- **KG-Threshold**: KG mode with entropy threshold as a secondary filter. If the KG predicts an answer but entropy is high, convert to "I don't know."
- **Relaxed**: Two-pass KG where the KG answer only overrides when the raw VLM answer disagrees. More permissive than strict KG mode.
- **Entropy**: Two-pass KG with entropy-based fusion. High entropy triggers KG override; low entropy trusts the VLM.
- **Raw Two-pass**: OVD detection plus VQA as a compute control. Runs both passes but does not use the KG for answer fusion. Measures detection overhead.

Chapter 4

Results

4.1 Prompt Engineering

The Optimized Equilibrium: Best Evaluation Results The best run was with *Qwen2.5-Coder-32B-Instruct*, as summarized in Table 4.1. It balanced *strictness* and *permissiveness*, achieving an epistemic equilibrium between rejecting distractors and accepting valid targets.

This configuration yielded the best results under local hardware constraints. The system completed 10 episodes in 57 minutes and 29 seconds (approximately 5.75 minutes per episode), with the detailed quantitative metrics reported in Table 4.1.

Metric	**Value**	**Analysis**
Success Rate ↑	0.2000	Significant improvement from 0%. The agent successfully identifies correct instances.
Distractor Success ↓	0.0000	False positives fully eliminated.
Target Detected ↑	0.5000	Vision stack successfully detects targets in half of episodes.
Stop Called ↑	0.4000	Cognitive layer confidently terminates search.
SPL ↑	0.0944	Indicates longer exploration paths in unknown environments.
Soft SPL ↑	0.2258	Agent moves toward correct regions even when failing.
Distance to Goal ↓	6.13 m	Remaining distance at termination.
Average Reward ↑	1.8931	Reflects overall navigation efficiency.

Table 4.1: Optimized CoIN-Bench evaluation results (`val_unseen`).

4.2 COCO-Based + IDKVQA VLM Fine-Tuning

This section reports the empirical results obtained from the fine-tuning pipeline. All values below are taken directly from the notebook outputs.

For the reported configuration (learning rate 3×10^{-5}, LoRA rank $r = 32$), validation accuracy improved monotonically across epochs:

Table 4.2: Validation performance across epochs

Epoch	Training loss	Validation accuracy
1	1.1981	21.2%
2	0.9787	25.9%
3	0.7908	27.1%
4	0.6300	29.4%

These results indicate stable optimization behavior under the COCO+IDKVQA mixed-training strategy, with consistent gains on the validation set over four epochs.

The final evaluation compared the baseline model and the best fine-tuned checkpoint on IDKVQA test data:

Table 4.3: Final comparison on IDKVQA test split

Model	Test accuracy	Best τ	Best ER
Baseline	31.0%	0.00	0.00
Fine-tuned (`lr3e-05_r32`)	34.5%	0.00	0.00

The fine-tuned model improved argmax test accuracy by +3.5 percentage points relative to baseline. Under the notebook's threshold sweep, the best entropy threshold remained $\tau = 0.00$ for both models, and the corresponding Effective Reliability (ER) was 0.00 in both cases.

4.3 COCO + Augmented IDKVQA VLM QLora Fine Tuning

The final metrics obtained with this pipeline are reported in Table 4.4, with the best threshold being $\tau = 0.86$, with 33 image transforms and 4 text templates per label.

Table 4.4: Final summary for `finetune_llava_idkvqa.py`

Metric	Value
Baseline accuracy (validation) ↑	47.1%
Fine-tuned accuracy (validation) ↑	56.5%
Validation improvement ↑	+9.4 percentage points
Test accuracy (argmax) ↑	69.8%
Effective Reliability (ER) ↑	70.7%

Our training curves (see C.1) show the behavior during fine tuning, reaching (in $\approx$ 6h) the best validation accuracy (early stopping had a patience of 2 epochs).

Our confusion matrices, comparing the baseline model to our finetuned version on the IDKVQA dataset are presented here. As we can see, the model's confirmation bias was mitigated. For more results, see C.2 and C.3.

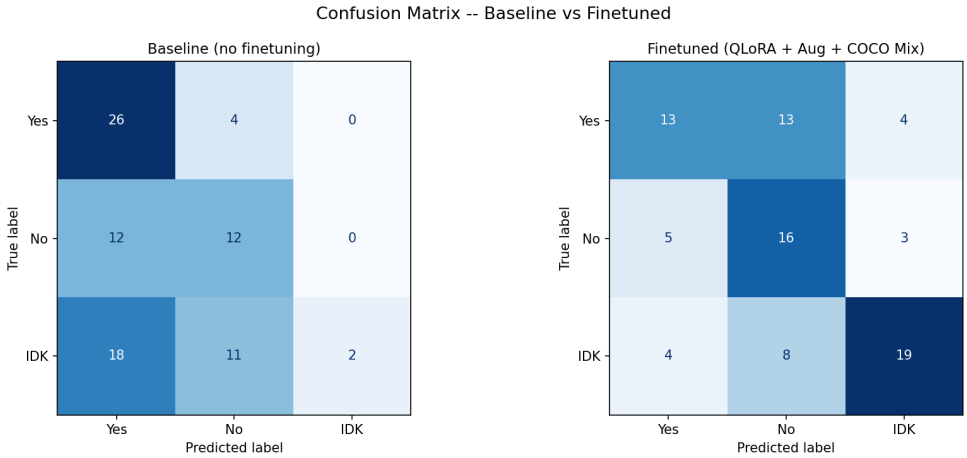

Figure 4.1: Confusion matrices for baseline and fine-tuned models.

4.4 AIUTA-VLM-R1

4.4.1 Offline Uncertainty Benchmarking

Before implementing the full CoIN framework with VLM-R1, we ran the uncertainty benchmark evaluated on IDKVQA dataset (502 samples, val split). The primary metric here is Effective Reliability, ϕ_{binary}, that penalizes confident wrong answers more than abstentions.

Mode	ϕ_{binary} (c=1)	Accuracy	Abstention Rate	Calls
Raw	21.71	53.6%	23.7%	1
Threshold $\tau = 0.10$	**33.27**	48.2%	43.4%	1
Threshold $\tau = 0.15$	32.87	51.0%	30.1%	1
KG	28.69	32.1%	91.6%	2–3
Two-pass KG	24.10	37.8%	59.4%	3
Relaxed	19.12	51.0%	30.1%	3
Entropy	23.90	48.6%	43.4%	3
Raw Two-pass	21.71	51.4%	17.9%	3

Table 4.5: Results comparison across different modes

For reference, the AIUTA paper (using LLaVA 7B + GPT-4o with 5–8 calls per sample) reports: MaxProb $\phi_{\text{binary}} = 15.94$ and NormalizedEntropy $\phi_{\text{binary}} = 21.12$. The thresholded result ($\tau = 0.10$) was tuned on the same 502-sample evaluation set; therefore, the fair comparison to AIUTA's $\phi_{\text{binary}} = 21.12$ is the raw (non-thresholded) setting.

Threshold-based filtering performs best, achieving the highest score with a simple single-pass approach, suggesting that VLM-R1's probability distribution provides a **well-calibrated uncertainty signal**. In contrast, **KG-based methods over-abstain** on single images due to insufficient extracted attributes, making them ineffective in this setting but potentially valuable in multi-step scenarios where information accumulates. Additionally, while the entropy computation differs from prior CoIN, it still preserves meaningful uncertainty ranking, supporting the effectiveness of thresholding.

4.4.2 Quantitative Results - VLM-R1

Then, we ran the benchmark included in the CoIN paper CoIN-Bench, that includes images from HM3D dataset. Table 4.6 summarizes the main performance differences between architectures. We measured the Number of Questions (NQ) per detection, the VLM/LLM calls, model size, Success Rate (SR), distance to goal (dtg) average and standard deviation.

Metric	AIUTA-VLM-R1 (Ours)	Original AIUTA
NQ per detection	3–5	3–4
Total VLM/LLM calls per detection	2–3	5–8
Model size	3B (single model)	7B + GPT-4o
SR (val_seen, 5 eps)	0	7.42
dtg range (m)	4.2 (2.2)	7.48 (2.88)

Table 4.6: Comparison between AIUTA-VLM-R1 and Original AIUTA

The zero success rate is attributable to the VLFM navigation stack, not the reasoning pipeline. It was identified through logs during the benchmark that the agent identifies the correct object through attribute matching but cannot navigate precisely enough to reach within 0.25m of the object center. The PointNav policy stops at its configured stop radius (0.75m), the depth-based goal estimation has significant error at typical detection distances, and the agent tends to orbit the target without converging.

Chapter 5

Discussions and Conclusion

5.1 Prompt Engineering

The initial low performance stemmed from pipeline bottlenecks rather than perception failures. Early prompts forced the autoregressive LLM to generate structured data before its reasoning, causing hallucinated attributes and frequent false positives. Reversing this order to enforce reasoning first fixed the hallucinations but caused the longer outputs to hit token limits. These truncated responses lacked closing tags, triggering parsing exceptions that paralyzed the agent. Additionally, brittle evaluation scripts crashed when attempting to aggregate missing metrics from these interrupted episodes.

Despite these engineering hurdles, the final benchmarks validated our methodology. Transitioning to strict, uncertainty-aware reasoning enabled true instance-level navigation. The resulting 0% distractor success rate confirms the optimized agent correctly rejects partial matches, highlighting the critical role of explicit uncertainty modeling.

However, this optimization process exposed the fragility of heavy zero-shot prompt engineering. Monolithic prompts requiring both deep reasoning and rigid formatting are computationally expensive and prone to syntax failures. Future systems must shift to decoupled pipelines, separating reasoning from data extraction to prevent token exhaustion. Furthermore, replacing manual string parsing with native function-calling APIs will guarantee format compliance. Ultimately, fine-tuning smaller models specifically for this targeted workflow will provide the fast, deterministic reliability required for embodied AI.

5.2 VLM Finetuning

Our results for the initial COCO + IDKVQA samples were disappointing, yielding values that, although increasingly better, seem too little to justify the costs. These results were what, in fact, motivated us, to pursue the QLora strategy with data augmentation. Simply put, it seems that the COCO dataset, although useful for providing more data, enabling us to properly finetune, only marginally increase model performance.

The results for the QLora approach, on the other hand, indicate a clear overall improvement after fine-tuning, both in validation accuracy and in final test-time decision quality under entropy-based thresholding. The confusion matrices and per-class plots show that the largest gain is observed on the *I don't know* class, which is critical for safe uncertainty-aware behavior in AIUTA. This gain is accompanied by a redistribution of performance across Yes/No classes, reflecting a more conservative and calibrated decision policy. Data augmentation seems to be a better target for fine tuning, which does make sense, given that it covers the original task's data distribution.

Qualitative inspection (see C.4) supports the quantitative findings: the model exhibits stronger abstention behavior in ambiguous scenes and improved consistency on visually challenging indoor observations.

Unfortunately we didn't have enough time to test the finetuned VLM directly on task, on the habitat simulator. The CoIN-Bench comparisons were let for the other two approaches for this reason.

5.3 VLM-R1 and Knowledge Graph Integration

The AIUTA-VLM-R1 approach presents a compelling architectural contribution: replacing the original dual-model stack (LLaVA 7B + GPT-4o, 5–8 calls per detection) with a single 3B-parameter VLM and a deterministic Knowledge Graph, reducing inference to 2–3 calls at zero API cost. The core insight is that VLM-R1's `<think>` blocks contain structured knowledge extractable through regex at no additional cost, replacing four LLM calls with deterministic KG operations — yields a fully interpretable and reproducible pipeline. However, the quantitative results on IDKVQA fell short of expectations. The raw baseline ($\varphi_1 = 21.71$) is only marginally above AIUTA's 21.12 on 502 samples, and the KG-augmented modes, which represent the actual contribution, performed worse: the conservative KG mode over-abstains (91.6% IDK rate), while relaxed and entropy fusion modes ($\varphi_1 = 19.12$ and 23.90) fall below and marginally above the raw baseline respectively. The efficiency gains are real, but the 3B model simply lacks the capacity to match a 7B+GPT-4o ensemble in decision quality.

Part of this gap is structural: the KG was designed to accumulate evidence over successive navigation steps, yet IDKVQA provides single static images with no temporal context; explaining the extreme conservatism. Whether the KG's cumulative value emerges in full online episodes on CoIN-Bench remains an open question we did not have time to validate at scale.

A notable limitation is the `QuestionGenerator`'s reliance on fixed templates, which restricts interaction richness compared to AIUTA's GPT-4o-generated questions. A promising direction would be leveraging the `<think>` block itself to generate contextual questions, recovering interaction quality without additional forward passes. Overall, the pipeline validates the feasibility of RL-trained reasoning combined with deterministic knowledge structures as an efficient alternative for embodied navigation, but future work must address the fusion logic, question generation quality, and crucially validate the KG's benefit in full online episodes.

Chapter 6

Contributions

Mohamed Amine led the literature review and state-of-the-art analysis, investigating navigation policies (VLFM, OpenFMNav, GAMap) and their compatibility with AIUTA. He benchmarked multiple LLMs (Llama-3-8B, Qwen2.5-Coder-32B, GPT-oss-20B) within the AIUTA pipeline, designed and executed the prompt engineering optimization that eliminated false positives, and conducted extensive experimental evaluation and debugging of the reasoning layer, including the diagnosis of the confirmation bias and autoregressive Chain-of-Thought trap. He also handled the system deployment on the computing cluster, including HM3D/CoIN-Bench dataset setup, multi-environment isolation (Habitat, LLaVA-NeXT, vLLM), GPU allocation across four H100s, and resolution of the dependency conflicts between LAVIS, Transformers, and NumPy.

Sergio Contente and Leonardo Falabella designed and implemented the AIUTA-VLM-R1 architecture: replacing the original multi-model stack (LLaVA 7B + GPT-4o) with a single 3B VLM and a deterministic Knowledge Graph. This included the full KG pipeline (TripleExtractor, GraphMatcher, QuestionGenerator, canonical alias dictionaries), the VLM-R1 detector and SelfQuestioner integration, and the Strategy-pattern component design with YAML-driven configuration. They implemented and evaluated eight ablation modes on the IDKVQA benchmark (502 samples), conducted entropy threshold sweeps, and produced the comparative analysis against the AIUTA baseline. They also integrated the pipeline into the VLFM navigation loop for CoIN-Bench online evaluation via the VLMr1Bridge and VLMr1Oracle modules, identifying and fixing ten iterative bugs across 265+ episodes.

Diego Fleury and Yasmin Van den Broek investigated the Akinator-inspired reasoning direction and prototypical learning approaches (ProtoCLIP, anchored embeddings) for structured hypothesis narrowing. They designed and executed the VLM fine-tuning experiments, including both the COCO+IDKVQA LoRA pipeline and the QLoRA augmented strategy with 33 image transforms and 4 text templates, achieving +9.4pp validation improvement and 70.7% Effective Reliability.

All members contributed to the report writing and documentation.

Appendix A

Study of the CoIN Framework and the AIUTA System

Before attempting to modify the underlying navigation policy or run novel experiments, it was imperative to conduct a rigorous structural and mathematical study of the baseline framework: CoIN (Collaborative Instance Navigation) and its core engine, AIUTA. Because all subsequent team tasks revolved around improving this paper's results, finding new research directions, and formulating research questions, documenting its internal mechanics, dependencies, and deployment process was a foundational step for the group.

A.1 Formal Definition of State Variables and Prompts

To understand AIUTA's dataflow, we must define its symbolic state. The agent initializes a fact set

$$F_0 = \{\text{“Find the picture”}\}.$$

At initialization, only the category is known-not a full grounded object description. At time t, the robot captures a visual observation O_t (an RGB image).

The dataflow relies on several prompt types:

- P_{init}: initial descriptive prompt to the VLM;
- P_{details}: prompt for generating or asking follow-up questions;
- P_{check}: Yes/No/Unknown verification prompt;
- $P_{\text{attribute}}$: prompt for attribute extraction or verification;
- P_{score}: prompt for alignment scoring and interaction trigger.

These generate intermediate semantic representations: S_{init} (initial caption), S_{enriched} (description augmented by self-dialogue), and S_{refined} (filtered description after uncertainty-aware verification).

A.2 The AIUTA Dataflow and Uncertainty Estimation

The AIUTA pipeline can be formally decomposed into the following sequential stages:

Stage A: Trigger from Navigation

When the Grounding-DINO detector emits an "OBJECT DETECTED!" signal, AIUTA is triggered. The semantic reasoning module does not run continuously over every frame; it is invoked only at semantically salient moments. This mirrors CoIN-style embodied agents, alternating between low-level action/perception and high-level language-mediated interpretation.

Stage B: Self-Questioner Enrichment

This phase transforms a raw observation into a richer symbolic description.

Initial Read. The VLM processes O_t with P_{init} to generate S_{init} (e.g., "There is a black-and-white picture on a light-blue wall..."). This compresses a visual latent into a structured natural language observation containing distinct attributes.

Hypothesis Expansion. An LLM generates follow-up questions

$$Q_{\text{details}} = \{q_1, q_2, \dots\}.$$

The VLM answers each question as

$$r_j = \text{VLM}(O_t, q_j).$$

Enrichment. The outputs are concatenated into

$$S_{\text{enriched}} = S_{\text{init}} + r_1 + r_2 + \cdots + r_j.$$

Functionally, this is a multi-turn self-dialogue transcript over the current scene.

Stage C: Uncertainty Estimation (The Core Innovation)

Instead of trusting raw VLM text blindly, AIUTA forces explicit uncertainty quantification from the VLM output distribution. Before attribute-level reasoning, the system asks a coarse verification question (P_{check}): "Does the image contain a picture? Answer with Yes, No, or I don't know."

Crucially, the system reads the probability distribution over the constrained vocabulary, for example:

$$p(\text{Yes}) = 0.85, \qquad p(\text{No}) = 0.10, \qquad p(\text{IDK}) = 0.05.$$

The uncertainty is computed via normalized Shannon entropy:

$$H = -\sum_i p(y_i) \log\big(p(y_i)\big) \tag{A.1}$$

Normalized by the maximum entropy for three outcomes,

$$H_{\max} = \log(3), \tag{A.2}$$

we get the uncertainty score:

$$u = \frac{H}{H_{\max}}. \tag{A.3}$$

If $u \leq \tau$ (where $\tau = 0.75$), the system is classified as "Certain" and proceeds. If $u > \tau$, the system is "Uncertain" and continues exploring. This separates the semantic answer from epistemic confidence, representing a major systems-engineering novelty.

Stage D: Attribute-Level Verification and Belief Filtering

If the coarse check is certain, an LLM extracts an attribute-value set

$$K = \{(k_1, v_1), \ldots, (k_n, v_n)\}$$

from S_{enriched}. The LLM generates specific attribute questions $Q_{\text{attribute}}$, and the VLM verifies each, returning an answer r_j and an uncertainty u_j using the exact same entropy formulation.

High-uncertainty attributes are explicitly removed. This means AIUTA acts as a belief filtering system. The resulting S_{refined} contains only attributes that survived uncertainty screening, converting a hallucination-prone caption into a trustworthy symbolic description suitable for downstream logic.

Stage E: Interaction Trigger Dataflow

Finally, the LLM receives S_{refined} and current facts F_t alongside an alignment prompt P_{score}. It outputs:

- s: a similarity score in $[0, 10]$ (or -1 for irrecoverable mismatch);
- $q_{a \to u}$: a potential question for the human user.

If the score indicates unresolved ambiguity, the system asks the user $q_{a \to u}$, and the fact state is dynamically updated:

$$F_{t+1} = \text{update}(F_t,\ \text{user answer}).$$

A.3 Information-Theoretic Interpretation and Novelty

From an architectural perspective, AIUTA is a sequence of typed maps:

$$(F_t, O_t) \to S_{\text{init}}$$
$$(F_t, O_t, S_{\text{init}}) \to Q_{\text{details}} \to \{r_j\} \to S_{\text{enriched}}$$
$$(O_t, P_{\text{check}}) \to p(y) \to u_{\text{check}}$$
$$(S_{\text{enriched}}, F_t) \to K \to Q_{\text{attribute}} \to \{(r_j, u_j)\} \to S_{\text{refined}}$$
$$(S_{\text{refined}}, F_t) \to (s, q_{a \to u})$$
$$(F_t, \text{ user answer}) \to F_{t+1}$$

While CoIN provides the template for iterative multimodal chain reasoning, AIUTA's primary novelty lies in its uncertainty-gated control flow and attribute-level uncertainty pruning. It translates raw VLM outputs into a verifiable semantic interface, asking targeted, highly contextual clarifying questions rather than generic ones.

A.4 Codebase Architecture and Dependency Graph

Understanding the dependencies of CoIN was a highly complex task, as it bridges 3D simulation, deep learning, and external APIs. We documented the repository into distinct dependency layers to facilitate debugging and deployment:

- **Simulation Core & Navigation:** Relies on `habitat-sim` (with Bullet physics for collisions) and `habitat-lab/habitat-baselines` for task definition and metric tracking (Success/SPL).
- **Core ML Stack:** Pinned to PyTorch v2.2.2 and Transformers v4.43.3 to prevent CUDA/ABI mismatches.
- **Vision/VLM Components:** Requires GroundingDINO for open-vocabulary detection, MobileSAM for segmentation masking, and Salesforce LAVIS (BLIP-2) for image-text matching.
- **Dataset Tooling:** Utilizes HuggingFace datasets to load the CoIN-Bench JSON splits, which define the evaluation episodes within the HM3D geometry.

A.5 Cluster Deployment and Technical Setup

Deploying the AIUTA system proved to be a significant engineering challenge. Initial attempts to install heavy models like LAVIS (BLIP-2) and compile GroundingDINO locally on laptops resulted in severe timeout issues and hardware limitations. Consequently, the deployment was migrated to the institution's computing cluster (ENSTA-l40s partitions via Slurm).

The cluster setup required meticulous environment management:

- **Headless Simulation:** `habitat-sim` was installed with the headless flag via `conda-forge` to allow 3D rendering on cluster nodes without connected displays.

- **CUDA Compilation:** GroundingDINO requires custom C++/CUDA extensions. A dedicated Slurm batch job (`sbatch`) was written to allocate a GPU and compile the extensions locally (`pip install -no-build-isolation -e .`), setting specific `TORCH_CUDA_ARCH_LIST` variables to match the cluster's NVIDIA architecture.

- **Dependency Resolution:** Significant PyTorch version conflicts were encountered and resolved by enforcing `torch==2.2.2` and `torchvision==0.17.2` configured for CUDA 12.1, ensuring that GroundingDINO CUDA operators loaded correctly.

- **Dataset Symlinking:** The HM3D dataset was mapped into the project directory using symbolic links (`ln -s`), preventing redundant downloads and optimizing storage usage.

By successfully deploying this stack, downloading the CoIN-Bench evaluation splits, and configuring the `COIN_LLM_CLIENT_KEY` (enabling seamless switching between OpenAI and Groq APIs), the team established a robust, reproducible environment capable of executing the VLM servers and running the `run_batch.sh` evaluation pipelines.

A.5.1 Metric Aggregation Failures and System Stability

After fixing the reasoning layer, the agent explored deeper environments, reaching the maximum step limit. This revealed a new failure: the evaluation script crashed with a `KeyError: 'yaw'`.

Cause: The aggregation loop assumed all episode logs contained identical keys, which was invalid due to varying termination conditions.

Fix: We replaced the brittle aggregation with safe filtering:

```
valid_values = [v[stat_key] for v in stats_episodes.values() if stat_key in v]
```

This ensured stable metric computation regardless of episode structure.

A.5.2 Evaluation Outcomes and the Over-Pruning Effect

With all fixes applied, the system became stable but exhibited complete task failure:

- Success rate: 0.0

- SPL: 0.0

- Soft-SPL: 0.2264

Diagnosis: The system suffered from *over-pruning*. The LLM, now strictly applying uncertainty filtering, removed too many attributes, producing overly sparse representations that never reached the acceptance threshold.

The non-zero Soft-SPL confirms that the agent navigated toward correct regions but failed to trigger the final success condition due to excessive conservatism.

Appendix B

AIUTA Dataflow Diagrams

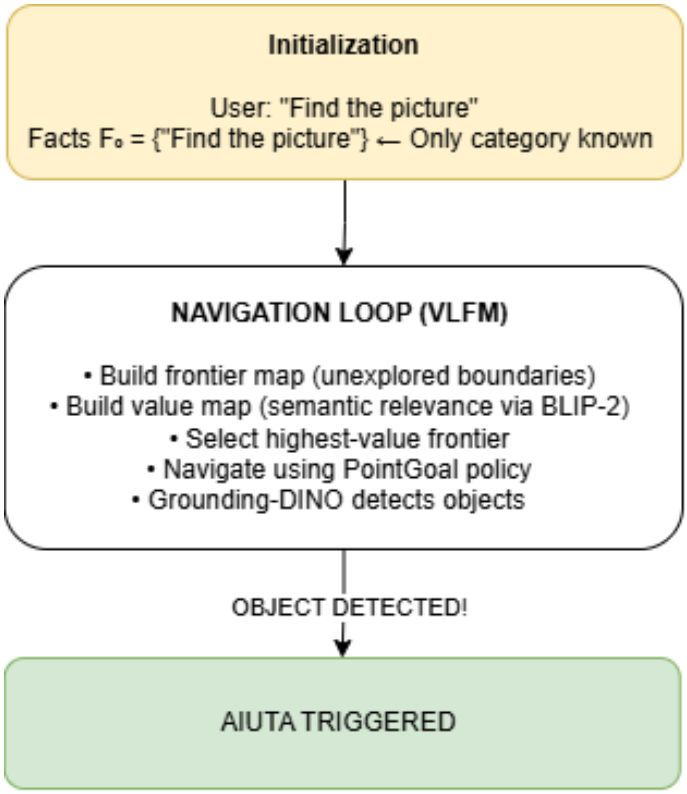

Figure B.1: Initialization phase and triggering of the AIUTA module.

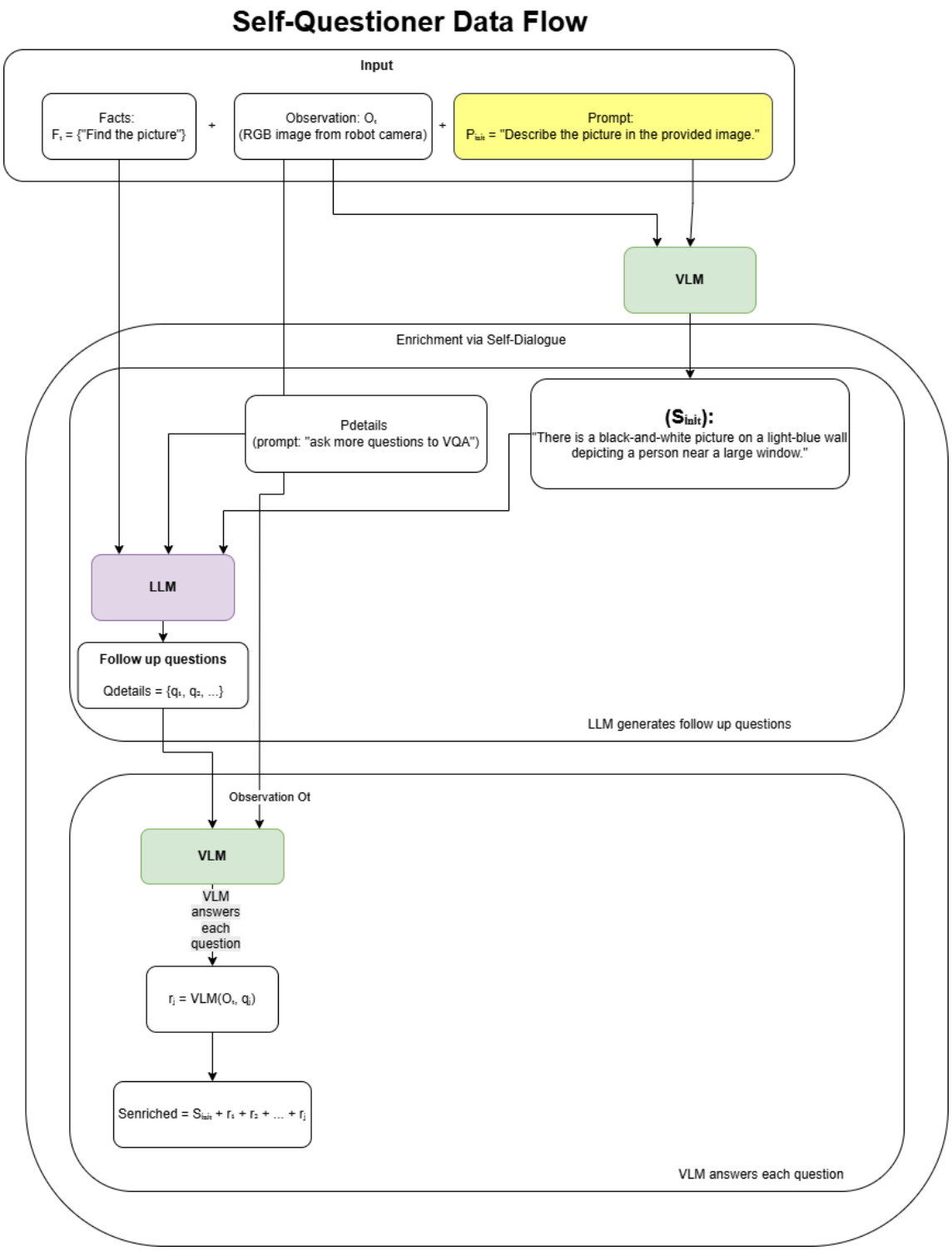

Figure B.2: Self-questioner data flow used for semantic enrichment.

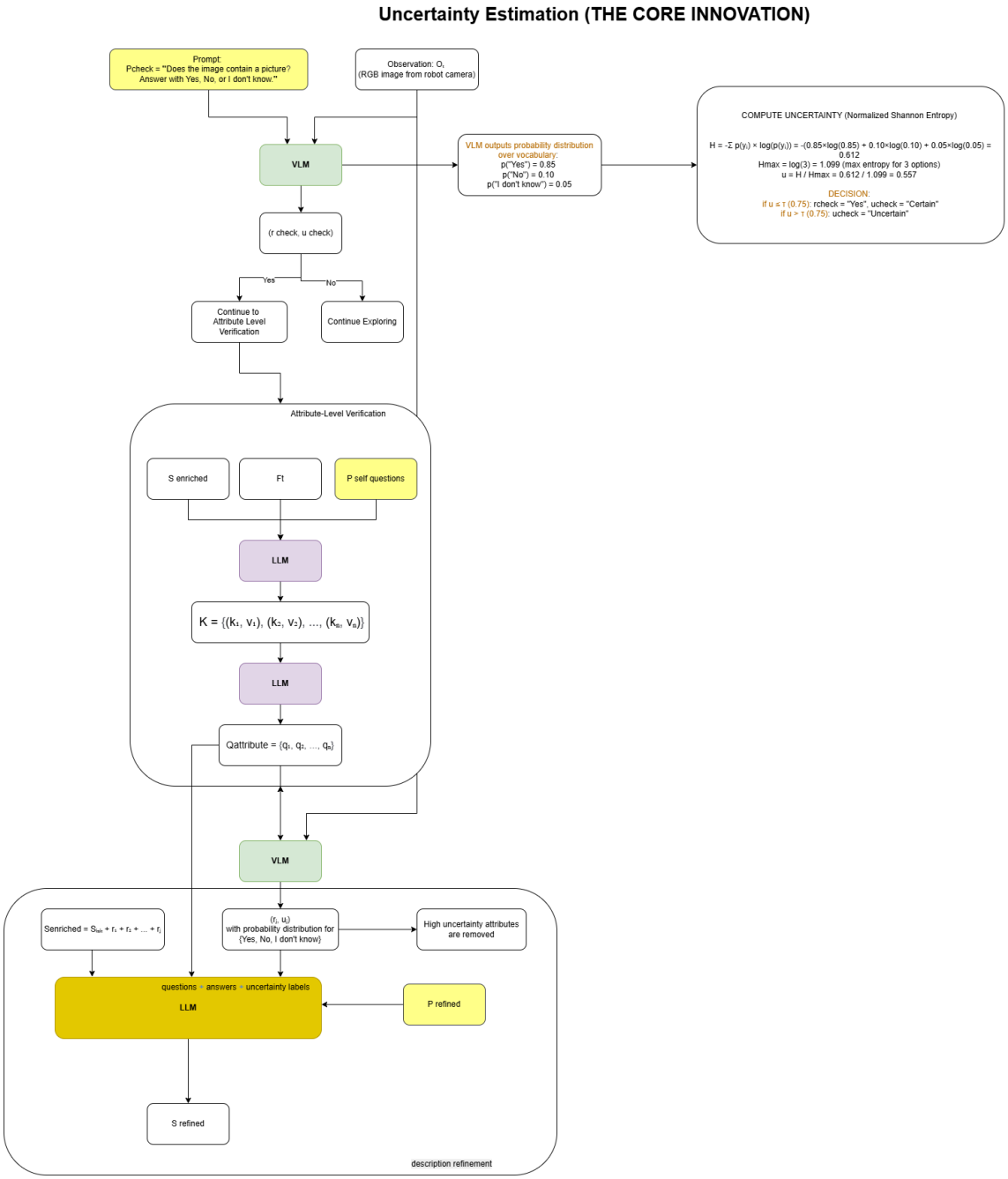

Figure B.3: Uncertainty estimation module based on normalized Shannon entropy (core innovation).

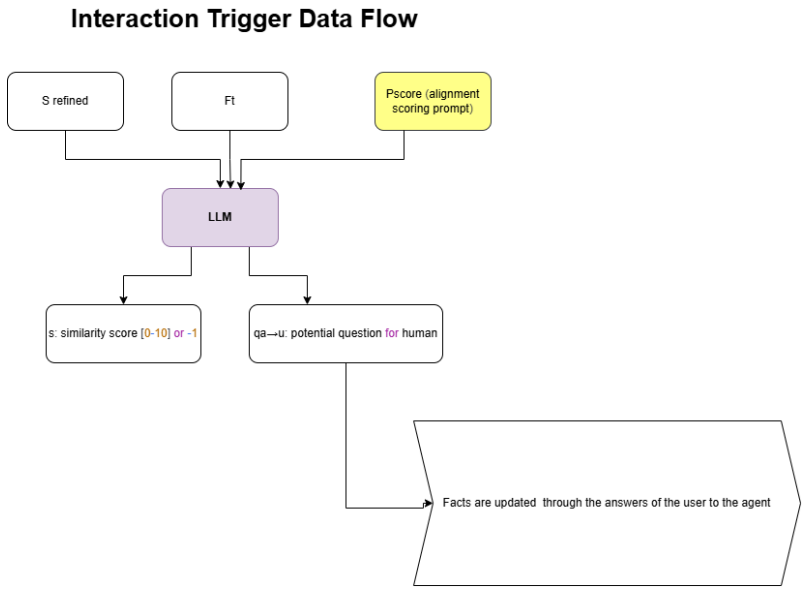

Figure B.4: Interaction trigger data flow and human-in-the-loop update mechanism.

Appendix C

VLM Finetuning experiments results

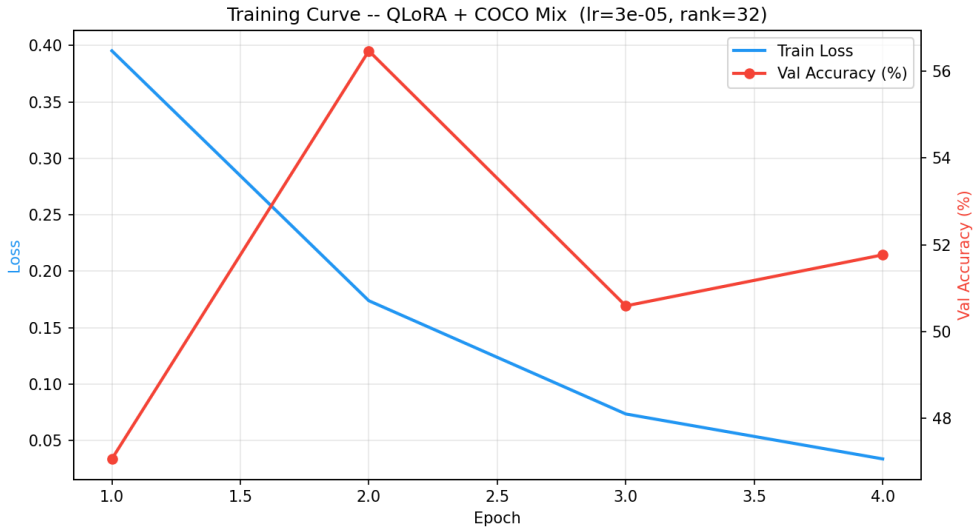

Figure C.1: Training dynamics of the QLoRA + COCO-mix pipeline (loss and validation accuracy).

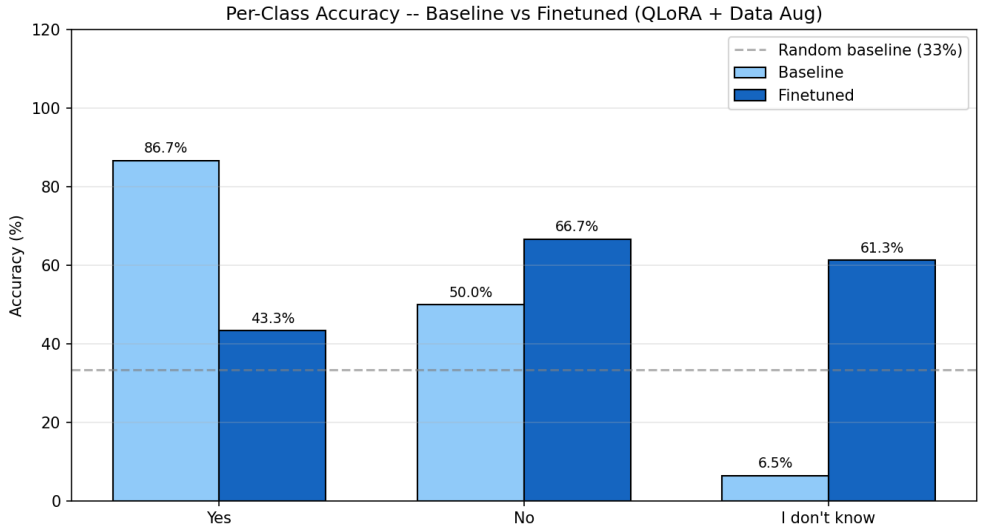

Figure C.2: Per-class accuracy comparison between baseline and fine-tuned models.

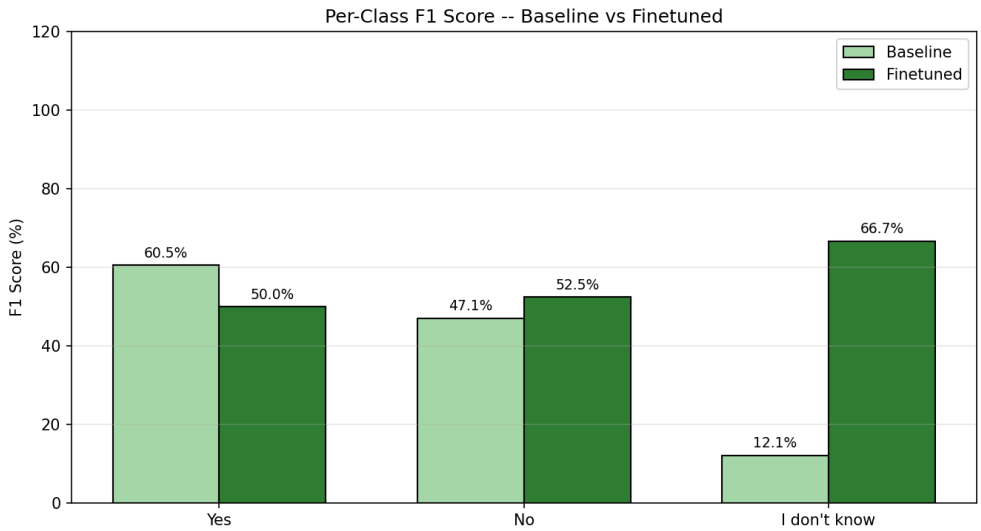

Figure C.3: Per-class F1-score comparison between baseline and fine-tuned models.

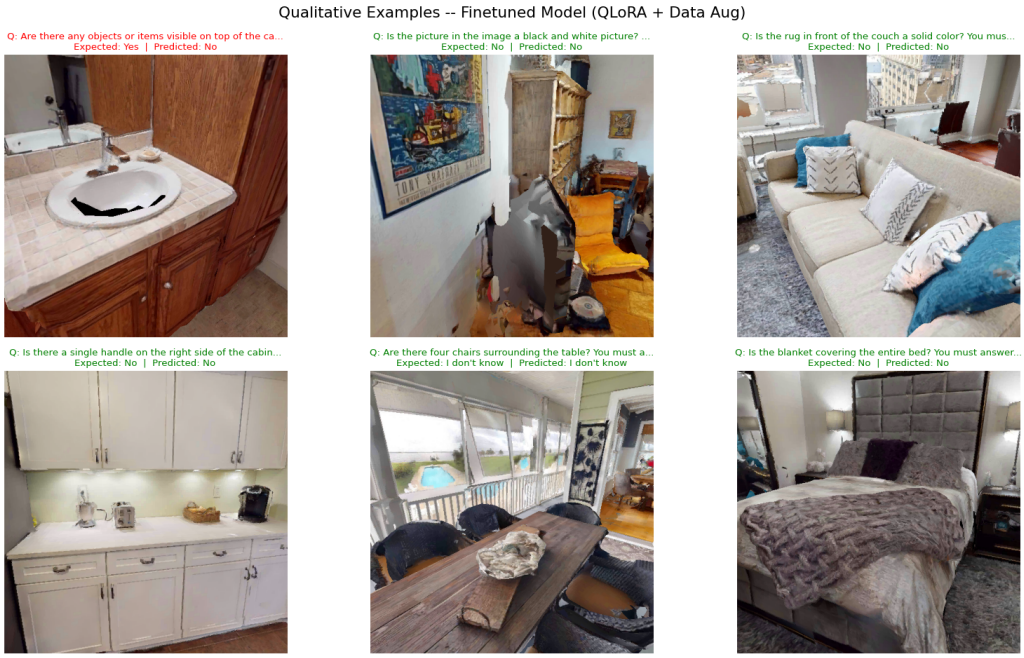

Figure C.4: Qualitative prediction examples from the fine-tuned model.

Appendix D

AIUTA-VLM-R1: Architechture and Dataflow diagrams

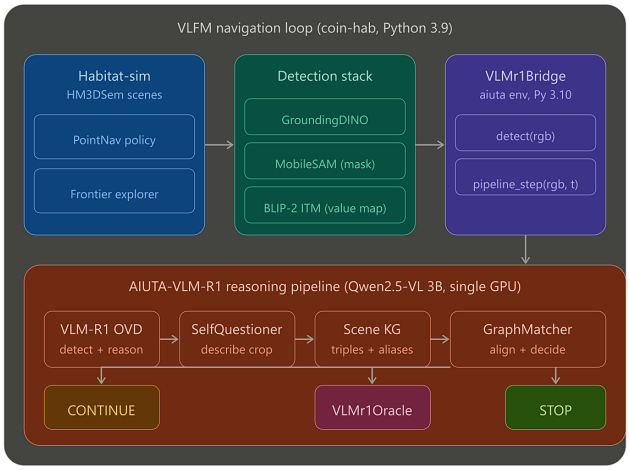

Figure D.1: AIUTA-VLM-R1 system overview.

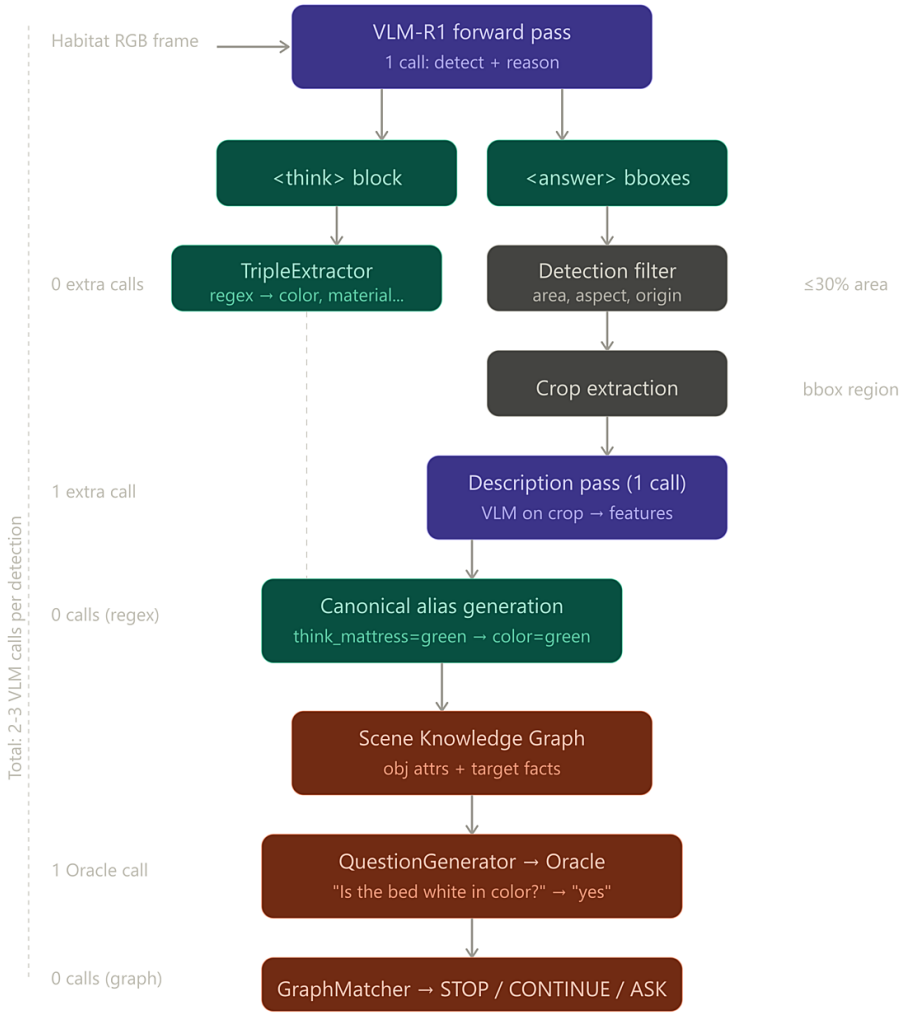

Figure D.2: AIUTA-VLM-R1 reasoning pipeline.

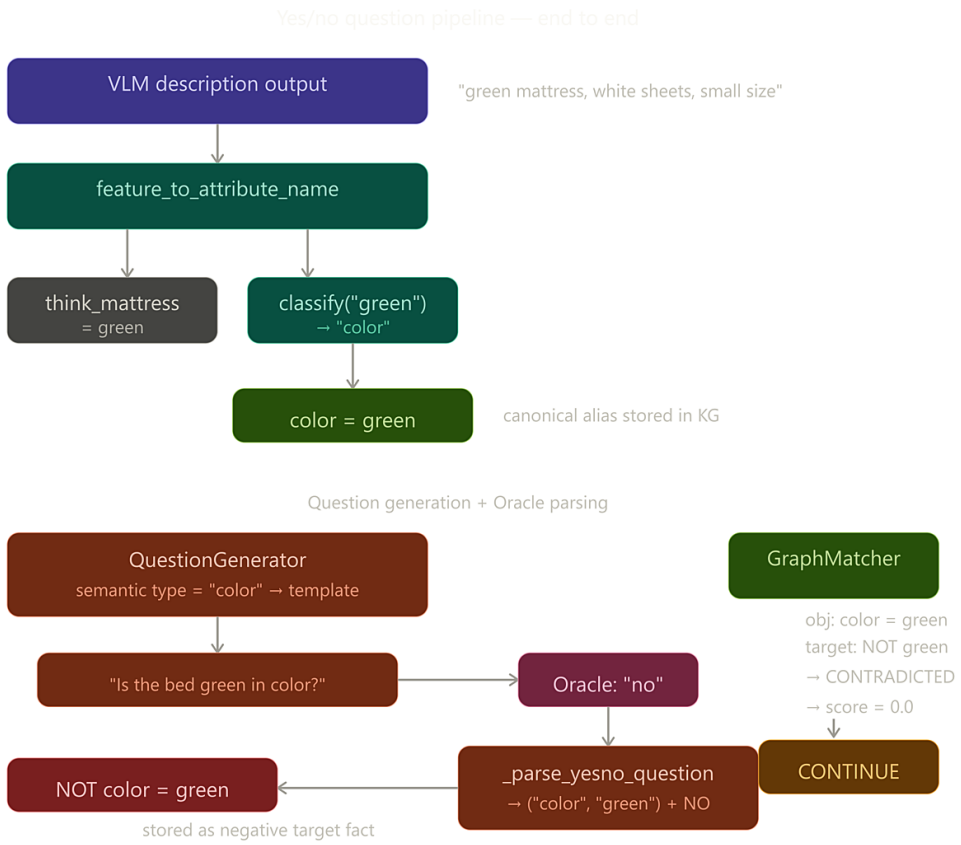

Figure D.3: Knowledge graph interaction flow with reasoning model's output.

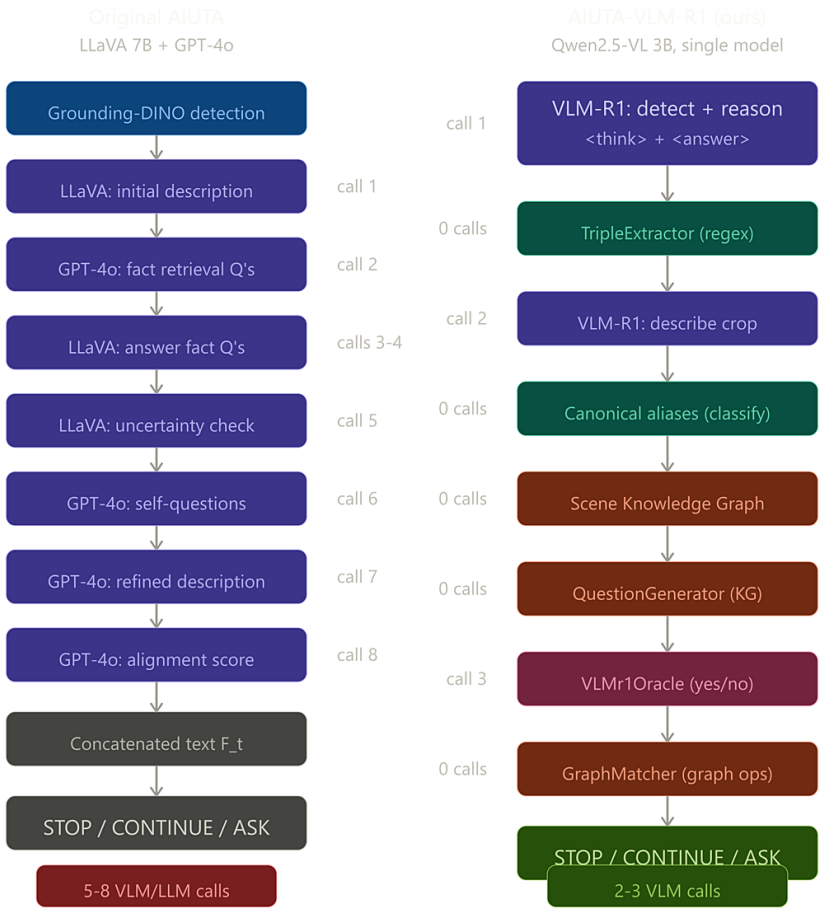

Figure D.4: VLM/LLM calls compairison between original AIUTA and proposed AIUTA-VLM-R1.

Bibliography

[1] Luca Andolfi and Eleonora Giunchiglia. Right for the right reasons: Avoiding reasoning shortcuts via prototypical neurosymbolic ai. Preprint, 2026. Emails: l.andolfi24@imperial.ac.uk, e.giunchiglia@imperial.ac.uk.

[2] Anonymous. Chatvla-2: Open-world vision-language-action reasoning for robotics. *arXiv preprint*, 2024.

[3] Tanmoy Bhattacharya, Aniruddha Kembhavi, Eric Kolve, Ali Farhadi, and Aniruddha Kembhavi. I don't know: Handling unanswerable questions in visual question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Aviral Fu, Sergio Guadarrama, et al. Rt-2: Vision-language-action models transfer knowledge to robotics. *Proceedings of the 7th Conference on Robot Learning (CoRL)*, 2023.

[5] Naralasetty Mounika Chandra mohan bhuma, sri harsha k, Surya Mounika Koneti, Harsha Vardhan Medisetti, and Siva Kumar Kurmala. Houseobj_vqa: A household object visual question answering dataset, 2025.

[6] Devendra Singh Chaplot, Dhiraj Gandhi, Saurabh Gupta, Abhinav Gupta, and Ruslan Salakhutdinov. Clip on wheels: Zero-shot object navigation as object localization and exploration. *arXiv preprint arXiv:2207.XXXXX*, 2022.

[7] X. Chen et al. Social navigation with vision-language models. *arXiv preprint*, 2023.

[8] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Pan Zhang, Bin Zhu, Lewei Lu, et al. Internvl: Scaling up vision-language foundation models and higher-resolution understanding. *arXiv preprint arXiv:2312.14238*, 2024.

[9] An-Chieh Cheng, Yandong Ji, Zhaojing Yang, Zaitian Gongye, Xueyan Zou, Jan Kautz, Erdem Bıyık, Hongxu Yin, Sifei Liu, and Xiaolong Wang. Navila: Legged robot vision-language-action model for navigation. *arXiv preprint arXiv:2402.XXXX*, 2024.

[10] Ekin D. Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V. Le. Autoaugment: Learning augmentation policies from data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[11] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[12] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aravind Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. PaLM-E: An embodied multimodal language model. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research (PMLR)*, pages 8469–8488, 2023.

[13] B. Yu et al. L3mvn: Leveraging large language models for visual target navigation. *arXiv preprint*, 2023.

[14] S. Yuan et al. Gamap: Zero-shot object goal navigation with multi-scale geometric-affordance guidance. *arXiv preprint*, 2024.

[15] Y. Kuang et al. Openfmnav: Towards open-set zero-shot object navigation via vision-language foundation models. *arXiv preprint*, 2024.

[16] G. Franchi A. Castellini A. Farinelli M. Cristani Y. Wang F. Taioli, E. Zorzi. Collaborative instance object navigation: Leveraging uncertainty-awareness to minimize human-agent dialogues. *arXiv preprint*, 2024.

[17] Daya Guo, Dejian Yang, Haowei Zhang, et al. Deepseek-r1 incentivizes reasoning in large language models through reinforcement learning. *Nature*, 2025.

[18] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[19] Moo Jin Kim et al. Openvla: An open vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[20] Moo Jin Kim, Karl Pertsch, Jiseung Oh, Suraj Singh, Soroush Nasiriany, Dhruv Shah, Huazhe Xu, Chelsea Finn, and Sergey Levine. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[21] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research (PMLR)*, pages 19730–19742, 2023.

[22] Y. Li et al. Language-guided mobile robot navigation with large language models. *arXiv preprint*, 2023.

[23] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision (ECCV)*, 2014.

[24] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, January 2024.

[25] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21253–21262, 2024.

[26] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jianwei Yang, Chunyuan Li, Jianfeng Yang, Hang Su, and Jun Zhu. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*, 2023.

[27] Arjun Majumdar, Gunjan Aggarwal, Bhavika Devnani, Judy Hoffman, and Dhruv Batra. ZSON: Zero-shot object-goal navigation using multimodal goal embeddings. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 32340–32352, 2022.

[28] J. J. P, K. Palanisamy, Y.-W. Chao, X. Du, and Y. Xiang. Proto-clip: Vision-language prototypical network for few-shot learning. *arXiv preprint*, 2024.

[29] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

[30] Haozhan Shen1 et al. Vlm-r1: A stable and generalizable r1-style large vision-language model. *arXiv preprint*, 2024.

[31] Lucy Xiaoyang Shi et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. *arXiv preprint arXiv:2403.XXXX*, 2024.

[32] Muhayy Ud Din, Waseem Akram, Lyes Saoud, Jan Rosell, and Irfan Hussain. Chatvla: Vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2401.XXXX*, 2024.

[33] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[34] Penghao Wu and Saining Xie. V*: Guided visual search as a core mechanism in multimodal llms, 2023.

[35] Naoki Yokoyama, Sehoon Ha, Dhruv Batra, Jiuguang Wang, and Bernadette Bucher. Vlfm: Vision-language frontier maps for zero-shot semantic navigation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[36] L et al. Zhang. Trihelper: Zero-shot object navigation with dynamic assistance. *arXiv preprint*, 2024.

[37] Mingjie Zhang, Yuheng Du, Chengkai Wu, Jinni Zhou, Zhenchao Qi, Jun Ma, and Boyu Zhou. Apexnav: An adaptive exploration strategy for zero-shot object navigation with target-centric semantic fusion. *IEEE Robotics and Automation Letters*, 2025.

[38] W et al. Zhang. Adaptive chain-of-focus reasoning via dynamic visual search and zooming for efficient vlms. *arXiv preprint*, 2024.